

Simulador de Memoria Virtual - sim-virtual

INF1316 - Sistemas Operacionais . Trabalho 2 (Gerencia de Memoria)

Integrantes: Caio Chaves | Algoritmos implementados: LRU, NRU, Relogio e Otimo

1. Resumo do projeto

O sim-virtual e um simulador de memoria virtual com paginacao, escrito em C. Ele recebe uma sequencia de acessos a memoria (enderecos logicos de 32 bits, cada um marcado como leitura R ou escrita W), realiza o mapeamento logico -> fisico por paginacao e simula a substituicao de paginas quando a memoria fisica esta cheia. Ao final, emite um relatorio com a configuracao usada, o numero de page-faults e o numero de paginas 'suja's que seriam escritas de volta no disco.

O programa recebe quatro argumentos de linha de comando, na ordem do enunciado:

```
sim-virtual <algoritmo> <arquivo.log> <tam_pagina_KB> <tam_memoria_MB>
exemplo: ./sim-virtual LRU arquivo.log 8 2
```

1.1. Do endereco logico a pagina

O indice da pagina e obtido descartando os s bits menos significativos do endereco: $page = addr \gg s$. O valor de s vem do tamanho da pagina (parametro c): $s = \log_2(tam_pagina_em_bytes)$. Assim, para 4 KB temos $s=12$ e para 8 KB, $s=13$. O numero de quadros e $num_frames = (memoria_MB * 2^{20}) / tam_pagina_bytes$.

1.2. Estruturas de dados

As estruturas centrais (definidas em sim-virtual.h) sao:

- Quadro fisico (Frame): para cada quadro guarda-se a pagina carregada, o bit R (referenciada), o bit M (modificada/suja) e o instante do ultimo acesso (last_access), exigidos pelo enunciado. Esses campos atendem a todos os algoritmos.
- Tabela de paginas (page_table): vetor indexado pelo numero da pagina que devolve o quadro onde ela reside, ou -1 se ausente. Como os enderecos tem 32 bits, ha $2^{(32-s)}$ paginas possiveis (ate ~1M entradas para 4 KB); custoso na pratica, mas aceitavel para a simulacao de um unico programa e com busca/insercao em $O(1)$.
- Vetor de acessos (Acesso[]): toda a sequencia do arquivo e carregada em memoria. Indispensavel para o algoritmo Otimo (que precisa 'ver o futuro') e usado de forma uniforme pelos demais.

Um contador time (iniciado em 0) e incrementado a cada acesso, simulando a passagem do tempo. Os page-faults (toda pagina carregada, inclusive o preenchimento inicial dos quadros) e as escritas de paginas sujas (somente quando uma pagina modificada e REMOVIDA) sao contabilizados. Paginas sujas ainda residentes ao final NAO contam como escritas, conforme o enunciado.

2. Algoritmos de substituicao implementados

2.1. LRU (Least Recently Used)

A cada acesso atualiza-se last_access = time do quadro. No page-fault com memoria cheia, remove-se a pagina com o MENOR last_access (usada ha mais tempo). Implementacao exata, $O(num_frames)$ por substituicao.

2.2. NRU (Not Recently Used)

Classifica cada quadro em uma de quatro classes a partir de (R, M): classe 0 ($R=0, M=0$), 1 ($R=0, M=1$), 2 ($R=1, M=0$) e 3 ($R=1, M=1$). A vitima e uma pagina da MENOR classe nao-vazia. Para que R reflita o uso recente, ele e zerado periodicamente, simulando a interrupcao de relógio do SO - aqui a cada INTERVALO_RESET_R (1000) acessos. O bit M so e limpo quando a pagina sai da

memoria.

2.3. Relógio (Clock / Second Chance)

Os quadros formam uma lista circular com um ponteiro. Na substituição, inspeciona-se o quadro sob o ponteiro: se $R=1$, dá-se 'segunda chance' (zera R e avança); se $R=0$, é a vítima. R é marcado a cada referência. É uma aproximação eficiente do LRU.

2.4. Ótimo (Belady)

Remove a página cujo PROXIMO uso está mais distante no futuro (ou que nunca mais será usada). Irrealizável na prática (exige conhecer o futuro), serve de limite inferior teórico de page-faults. Para eficiência, pre-computamos, para cada página, a lista ordenada dos índices em que ela é acessada (estrutura 'CSR': `occ_flat` achatado, com `occ_ini/occ_fim` delimitando o bloco de cada página e `occ_ptr` na ocorrência atual); assim o 'próximo uso' de qualquer página residente sai em $O(1)$.

2.5. Pseudo-código do algoritmo Ótimo

```
funcao Otimo(sequencia A[0..n-1], num_frames):
    # Pre-processamento: para cada página, a fila de índices em que aparece
    para i de 0 ate n-1:
        p = pagina(A[i]); enfileira i em occ[p]

    quadros = {}; faults = 0; escritas = 0
    para time de 0 ate n-1:
        p = pagina(A[time]); escrita = (A[time] e 'W')
        desenfileira o índice 'time' de occ[p] # avança p p/ próximo uso

        se p está em quadros:
            marca R, M (se escrita) de p # acerto, sem falta
            continua

        faults = faults + 1 # page-fault
        se há quadro livre:
            carrega p em um quadro livre
        senão:
            # vítima = página com o MAIOR próximo uso
            vítima = página cujo próximo uso (frente de occ[.]) é o MAIOR
                    (occ[.] vazio => próximo uso = +infinito = nunca mais)
            se vítima está suja (M=1): escritas = escritas + 1
            remove vítima dos quadros; carrega p no lugar da vítima
    retorna (faults, escritas)
```

3. Resultados das simulacoes

As quatro cargas reais (compilador, matriz, compressor e simulador), cada uma com 1.000.000 de acessos, foram simuladas para todas as combinacoes de algoritmo (LRU/NRU/Relogio/Otimo), tamanho de pagina (4 KB e 8 KB) e memoria (1, 2 e 4 MB) - 24 simulacoes por programa, 96 no total. A varredura foi automatizada pelo script run_all.sh, que gera o arquivo resultados.csv. As tabelas abaixo trazem, para cada programa, o numero de page-faults (PF) e de paginas escritas (PE).

3.1. compilador.log

Pag (KB)	Mem (MB)	LRU PF	LRU PE	NRU PF	NRU PE	Relogio PF	Relogio PE	Otimo PF	Otimo PE
4	1	25,308	4,231	41,819	3,379	26,789	4,528	12,667	2,483
4	2	10,425	2,173	38,178	2,282	11,472	2,398	5,662	1,328
4	4	4,391	955	22,559	239	4,765	1,086	3,047	726
8	1	36,707	5,775	41,975	4,464	38,827	6,250	21,271	3,711
8	2	21,091	3,839	38,805	3,313	22,655	4,162	10,118	2,190
8	4	7,632	1,756	32,604	1,687	8,412	1,981	4,232	1,078

3.2. matriz.log

Pag (KB)	Mem (MB)	LRU PF	LRU PE	NRU PF	NRU PE	Relogio PF	Relogio PE	Otimo PF	Otimo PE
4	1	5,673	1,866	16,410	1,901	5,980	1,973	3,572	1,267
4	2	3,632	1,159	13,863	1,283	3,795	1,218	2,672	985
4	4	2,803	789	9,133	276	2,901	809	2,543	672
8	1	10,928	3,273	17,373	2,470	12,896	3,844	5,361	1,827
8	2	4,745	1,623	14,764	1,652	4,870	1,676	2,969	1,109
8	4	2,950	985	12,175	1,072	3,064	1,011	2,295	843

3.3. compressor.log

Pag (KB)	Mem (MB)	LRU PF	LRU PE	NRU PF	NRU PE	Relogio PF	Relogio PE	Otimo PF	Otimo PE
4	1	397	48	595	0	432	71	317	36
4	2	317	0	317	0	317	0	317	0
4	4	317	0	317	0	317	0	317	0
8	1	533	132	916	18	577	163	345	86
8	2	255	0	255	0	255	0	255	0
8	4	255	0	255	0	255	0	255	0

3.4. simulador.log

Pag (KB)	Mem (MB)	LRU PF	LRU PE	NRU PF	NRU PE	Relogio PF	Relogio PE	Otimo PF	Otimo PE
4	1	11,240	4,092	25,906	5,127	11,876	4,319	5,981	2,575
4	2	5,823	2,444	24,154	4,385	6,119	2,587	4,125	1,884
4	4	4,468	1,846	18,277	2,146	4,618	1,937	3,890	1,565
8	1	16,479	5,546	24,885	6,002	17,923	6,070	9,179	3,528
8	2	8,556	3,395	22,747	4,616	9,140	3,666	4,748	2,160
8	4	4,732	2,094	20,944	3,952	4,875	2,171	3,498	1,620

PF = page-faults (paginas carregadas). PE = paginas escritas (paginas sujas removidas).

4. Analise de desempenho

4.1. Comparacao entre os algoritmos de substituicao

Em TODAS as 96 simulacoes o Otimo apresentou o menor numero de page-faults, confirmando seu papel de limite inferior teorico. Por exemplo, em compilador com 4 KB e 1 MB: Otimo 12.667 contra LRU 25.308, Relogio 26.789 e NRU 41.819 - o Otimo gera cerca de metade das faltas do LRU.

Entre os algoritmos realizaveis, LRU e Relogio ficam sempre muito proximos, com o Relogio ligeiramente pior (ele aproxima o LRU usando apenas o bit R). O NRU foi consistentemente o pior em page-faults - a granularidade de apenas quatro classes e a dependencia do periodo de reset do bit R o tornam grosseiro. Em contrapartida, o NRU produziu sistematicamente o MENOR numero de escritas de paginas sujas, porque prioriza remover paginas nao-modificadas (classes 0 e 2 antes de 1 e 3). O caso mais marcante e compilador 4 KB / 4 MB: NRU escreve apenas 239 paginas, contra 955 do LRU e 1.086 do Relogio. Ou seja, o NRU troca mais faltas por menos escritas de volta ao disco.

4.2. Efeito do tamanho da pagina (memoria fixa)

Mantendo a memoria fisica fixa, dobrar o tamanho da pagina (de 4 para 8 KB) reduz pela metade o numero de quadros disponiveis, mas cada pagina cobre o dobro do espaco de enderecos. O efeito liquido depende da localidade espacial da carga:

- compilador, matriz e simulador: o numero de page-faults AUMENTA com paginas de 8 KB. Ex.: matriz/LRU/1 MB passa de 5.673 (4 KB) para 10.928 (8 KB); compilador/LRU/1 MB de 25.308 para 36.707. Para essas cargas, a perda de metade dos quadros pesa mais do que o ganho de cobertura, ou seja, a localidade espacial nao e forte o bastante para compensar - mesmo no matriz, cujo acesso a matrizes nao e suficientemente sequencial dentro de uma pagina maior.
- compressor: aqui o working set e pequeno e cabe na memoria a partir de 2 MB; as faltas passam a ser dominadas pelas faltas compulsorias (primeira carga de cada pagina). Como paginas maiores significam menos paginas distintas para cobrir o mesmo working set, o 8 KB gera MENOS faltas (255 contra 317 em 2 MB e 4 MB).

Quanto as escritas, paginas maiores tendem a aumentar o numero de paginas sujas removidas (uma pagina maior tem mais chance de conter algum endereco escrito): ex. compilador/LRU/4 MB passa de 955 (4 KB) para 1.756 (8 KB).

4.3. Efeito do tamanho da memoria fisica

Aumentar a memoria (mais quadros) reduz monotonicamente os page-faults de todos os algoritmos, em todos os programas. Ex.: compilador/LRU/4 KB cai de 25.308 (1 MB) para 10.425 (2 MB) e 4.391 (4 MB). Quando o working set inteiro cabe na memoria (caso do compressor a partir de 2 MB), restam apenas as faltas compulsorias e o numero de escritas vai a zero, pois nenhuma pagina precisa ser removida.

4.4. Conclusao

Os resultados seguem o esperado pela teoria: Otimo < LRU ~ Relogio < NRU em page-faults; o NRU compensa com menos escritas ao disco; mais memoria sempre ajuda; e, para estas cargas, paginas maiores em geral pioram o desempenho (exceto quando o working set ja cabe na memoria). O LRU mostrou-se o melhor algoritmo realizavel em quase todos os cenarios, com o Relogio como alternativa de implementacao mais barata e desempenho praticamente equivalente.

5. Como compilar e executar

```
make # compila o sim-virtual
./sim-virtual LRU compilador.log 8 2
```

```
# varredura completa (com os .log no diretorio):  
./run_all.sh          # gera resultados.csv com as 24 combinacoes/arquivo
```

Validacao: as saidas dos quatro algoritmos foram comparadas com uma implementacao de referencia independente em Python sobre uma carga aleatoria; os contadores de page-faults e de paginas escritas coincidiram exatamente em todos os casos.